

addEventListener()

The `addEventListener()` method is used to attach an event to an element.

Syntax :

`element.addEventListener(event, function, useCapture);`

- `element` - The HTML element to which the event is attached.
- `event` - The type of event (`click`, `mouseover`, `keydown`, etc.)
- `function` - The function to be executed when the event occurs.
- `useCapture` - Optional. A Boolean value (`true` or `false`).

Example :

```
let btn = document.getElementById("myBtn");  
btn.addEventListener("click", function() {  
    alert("Button clicked!");  
});
```

When the button is clicked, the alert box will appear.

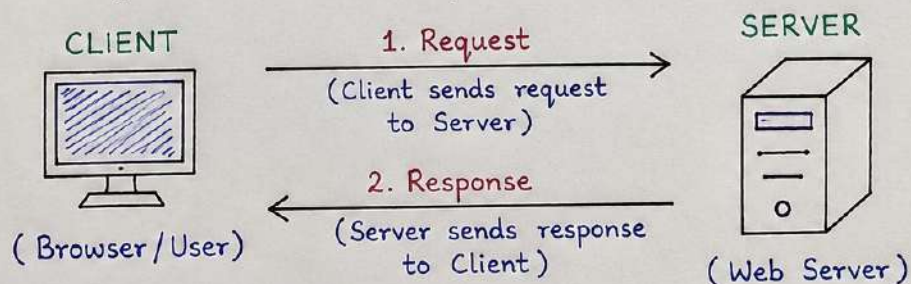
Server-Side vs Client-Side

1) Server-Side

- Code is executed on the **server**.
- The server processes the request, performs operations (like database access) and sends the result to the client.
- More **secure** (sensitive data stays on server).
- **Examples** : PHP, Python, Java, Node.js, ASP.NET

2) Client-Side

- Code is executed on the **client's** browser.
- The browser processes the code and shows the result to the user.
- Less **secure** (code is visible to user).
- **Examples** : HTML, CSS, JavaScript (runs in browser)



Q1. Explain the difference between var, let, and const with examples.

Answer:

The main differences between var, let, and const are as follows:

Feature	var	let	const
Scope	Function scoped	Block scoped	Block scoped
Hoisting	Hoisted and initialized with undefined	Hoisted but cannot be accessed before declaration (Temporal Dead Zone)	Hoisted but cannot be accessed before declaration (Temporal Dead Zone)
Re-declaration	Allowed	Not allowed	Not allowed
Re-assignment	Allowed	Allowed	Not allowed
Initialization	Optional	Optional	Mandatory
Use	Old way (avoid in modern JavaScript)	Use when value may change	Use when value should not change

Examples:

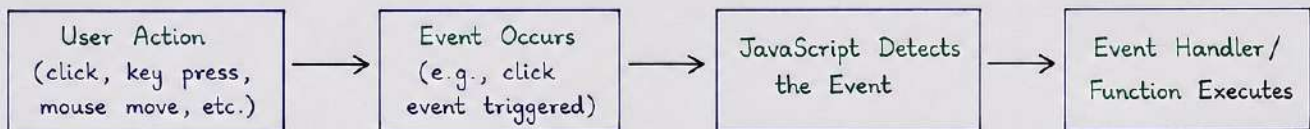
var	let	const
<pre>console.log(a); // undefined var a = 10; a = 20; console.log(a); // 20 var a = 30; // re-declaration allowed console.log(a); // 30</pre>	<pre>// console.log(b); // ReferenceError let b = 10; b = 20; console.log(b); // 20 // let b = 30; // Error: Cannot // redeclare</pre>	<pre>// console.log(c); // ReferenceError const c = 10; // c = 20; // Error: Assignment to // constant variable // const c = 30; // Error: Cannot // redeclare</pre>

Note: Use const by default. Use let when value needs to change.
Avoid var in modern JavaScript.

Q2. How does JavaScript handle user events? Explain with examples.

Answer: JavaScript handles user events by listening for actions performed by the user, such as clicking a button, pressing a key, moving the mouse, etc., and then executing a function (**event handler**) in response to that action.

How it works:



Examples:

1) Click Event

```
<button id="btn"> Click Me</button>
```

```
document.getElementById('btn')  
.addEventListener('click', function(){  
    alert("Button was clicked!");  
});
```

When the button is clicked, the alert box will appear.

2) Keydown Event

```
<input type="text" id="txt" placeholder="Type something">
```

```
document.getElementById('txt')  
.addEventListener('keydown', function(event){  
    console.log("Key pressed: " + event.key);  
});
```

When a key is pressed while typing, the pressed key is shown in console.

3) Mouse Over Event

```
<p id="para" style="color:blue;">Hover over me</p>
```

```
document.getElementById('para')  
.addEventListener('mouseover', function(){  
    this.style.color = "red";  
});
```

When the mouse pointer comes over the paragraph, the text color changes to red.

Common Events:

- **Mouse Events** – click, dblclick, mouseover, mouseout, mousemove
- **Keyboard Events** – keydown, keyup, keypress
- **Form Events** – submit, change, focus, blur

Key Point:

JavaScript uses `addEventListener()` method to attach an event handler to an element.

3. What is the difference between == and === in JavaScript?

Basis	== (Equality Operator)	=== (Strict Equality Operator)
Comparison	Compares only values	Compares both value and data type
Type Conversion	Performs automatic type conversion	Does not perform type conversion
Accuracy	Less strict	More strict and accurate
Result	May return true even if data types are different	Returns true only if both value and data type are the same
Best Use	Rarely used	Recommended in modern JavaScript
Example	5 == "5" → true	5 === "5" → false 5 === 5 → true

Q4. Explain JavaScript functions and their different types.

Answer: A function in JavaScript is a block of code that performs a specific task. Functions help in code reusability, better organization and easy maintenance.

Syntax:

```
function functionName(parameters){  
    // code to be executed  
    return value;    // optional  
}
```

Example:

```
function add(a,b){  
    return a + b;  
}  
  
let sum = add(5,3);    // 8
```

Types of JavaScript Functions:

1) Function Declaration

A function declared using the function keyword.

```
function greet(){  
    console.log("Hello!");  
}  
  
greet();
```

Hoisted - can be called before declaration.

2) Function Expression

A function stored in a variable.

```
const greet = function(){  
    console.log("Hello!");  
};  
  
greet();
```

Not hoisted - cannot be called before declaration.

3) Arrow Function

A shorter way to write functions using `=>`.

```
const greet = () => {  
    console.log("Hello!");  
};  
  
greet();
```

Shorter syntax and does not have its own this.

4) Anonymous Function

A function without a name.

```
setTimeout(function(){  
    console.log("Hi!");  
}, 1000);
```

Used when a function is passed as an argument.

5) Immediately Invoked Function Expression (IIFE)

A function that runs as soon as it is defined.

```
(function(){  
    console.log("IIFE runs immediately");  
})();
```

Used to avoid polluting global scope.

6) Constructor Function

Used to create objects using the new keyword.

```
function Person(name, age){  
    this.name = name;  
    this.age = age;  
}
```

```
const p1 = new Person("Alice", 20);  
  
console.log(p1.name);    // Alice
```

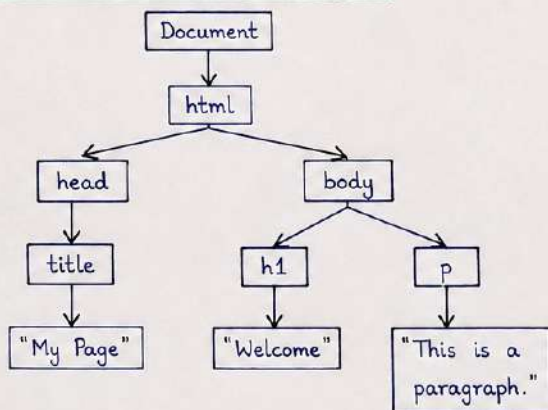
Key Points:

- Functions make code modular and reusable.
- They can take parameters and return values.
- Different types are used based on the need.

Q5. What is the Document Object Model (DOM)? Explain its importance.

Answer: The Document Object Model (DOM) is a programming interface for web documents. It represents the structure of an HTML or XML document as a tree of objects, where each object is a node that can be accessed and modified using JavaScript.

DOM Tree Structure (Example):



Importance of DOM:

- 1) It represents the web page as a tree structure, making it easy to access and update any element.
- 2) It allows JavaScript to dynamically change the content, structure and style of a web page.
- 3) It enables event handling, e.g., clicking a button, submitting a form, etc.
- 4) It improves interactivity and provides a better user experience.

DOM Manipulation Example:

```
// HTML : <p id="msg">Hello</p>
let para = document.getElementById("msg");
para.innerHTML = "Hello, DOM!"; // Change content
para.style.color = "blue";      // Change style
```

In Short:

DOM acts as a bridge between HTML/CSS and JavaScript. It allows JavaScript to read, modify and interact with the elements of a web page.

6. Explain the Window Object and its commonly used methods.

Answer: The Window object represents an open window in a browser. It is a global object in JavaScript with many properties and methods. All global variables, functions and objects automatically become members of the Window object.

Commonly Used Methods of Window Object:

Method	Description	Example
alert()	Displays an alert box with a message and an OK button.	<code>alert("Welcome!");</code> // shows an alert box
confirm()	Displays a dialog box with a message and OK and Cancel buttons. Returns true if OK is clicked, otherwise false.	<code>let result = confirm("Do you want to exit?");</code> // returns true or false
prompt()	Displays a dialog box that prompts the user for input. Returns the input value or null if Cancel is clicked.	<code>let name = prompt("Enter your name:");</code> // returns user input or null
open()	Opens a new browser window.	<code>window.open("https://www.google.com");</code> // opens a new window
close()	Closes the current window.	<code>window.close();</code> // closes the window
setTimeout()	Executes a function or code after a specified time (in milliseconds).	<code>setTimeout(function(){ alert("Hello"); }, 3000);</code> // runs after 3000 ms (3 seconds)
setInterval()	Executes a function or code repeatedly at a specified time interval.	<code>setInterval(function(){ console.log("Hi"); }, 2000);</code> // runs every 2000 ms (2 seconds)
location	Provides information about the current URL and is used for navigation.	<code>window.location.href = "https://example.com";</code> // redirects to new URL

Example:

```
function greet(){  
    alert("Hello, Welcome to JavaScript!");  
}  
window.setTimeout(greet, 2000); // calls greet() after 2 sec
```

Note:

The Window object can be written as "window". All properties and methods can be accessed using the window object.

7. What are JavaScript Objects? Explain with an example.

Answer: In JavaScript, an object is a collection of key-value pairs. It is used to store related data and more complex entities. Objects are one of the fundamental data types in JavaScript.

Key Points:

- Keys are always strings (or symbols).
- Values can be any data type (number, string, boolean, array, function, another object, etc.).
- Objects are mutable (values can be changed).

Example:

```
// Creating an object
let student = {
  name : "Ayush",
  age : 20,
  course : "B.Sc. IT",
  isStudent = true,
  skills : ["HTML", "CSS", "JavaScript"],
  greet : function(){
    console.log("Hello! I am " + this.name);
  }
};
```

Accessing Object Properties:

Using dot notation

```
console.log(student.name); // Ayush
console.log(student.age); // 20
student.course = "M.Sc. IT"; // update
```

Using bracket notation

```
console.log(student["name"]); // Ayush
console.log(student["age"]); // 20
student["isStudent"] = false; // update
```

Using Object Methods:

```
student.greet();
// Output: Hello! I am Ayush
```

Explanation:

Here, "student" is an object. It has several properties like name, age, course, etc., and a method "greet()". We can access and modify these properties using dot notation or bracket notation.

8. Explain the different looping statements available in JavaScript.

Answer: Looping statements are used to execute a block of code repeatedly until a specified condition is met.

Types of Looping Statements in JavaScript:

1) for Loop

Used when the number of iterations is known.

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}  
  
// Output: 1 2 3 4 5
```

How it works: Initialization → Condition
→ Execution → Update

2) while Loop

Repeats a block of code while the condition is true.

```
let i = 1;  
while (i <= 5) {  
  console.log(i);  
  i++;  
}  
  
// Output: 1 2 3 4 5
```

3) do...while Loop

Similar to while loop, but the block executes at least once.

```
let i = 1;  
do {  
  console.log(i);  
  i++;  
} while (i <= 5);  
  
// Output: 1 2 3 4 5
```

4) for...in Loop

Used to iterate over the enumerable properties (keys) of an object.

```
let person = { name: "Ayush", age: 20 };  
for (let key in person) {  
  console.log(key + ":" + person[key]);  
}  
  
// Output: name : Ayush    age : 20
```

5) for...of Loop

Used to iterate over the values of an iterable object like array, string, etc.

```
let fruits = ["Apple", "Banana", "Mango"];  
for (let fruit of fruits) {  
  console.log(fruit);  
}  
  
// Output: Apple Banana Mango
```

Key Points:

- Use for loop when the number of iterations is known.
- Use while loop when the number of iterations is unknown.
- Use do...while loop when the block must execute at least once.
- Use for...in to loop through object keys.
- Use for...of to loop through values of iterables (arrays, strings, etc.).

9. What is event bubbling and event capturing in JavaScript?

Answer: When an event occurs on an element in JavaScript, the event is first captured by the outer (parent) elements and then reaches the target element. After that, it bubbles back up from the target to the outer elements. This whole process is called **Event Propagation**.

1) Event Bubbling (Default)

- In bubbling, the event starts from the target element and bubbles up to its ancestors (parent → grandparent → ...).
- This is the default behavior in JavaScript.

```
<div id="grandparent">
  <div id="parent">
    <button id="child">Click Me</button>
  </div>
</div>
```

```
<script>
grandparent.addEventListener("click", () =>
  console.log("Grandparent"));
parent.addEventListener("click", () =>
  console.log("Parent"));
child.addEventListener("click", () =>
  console.log("Child"));
</script>
```

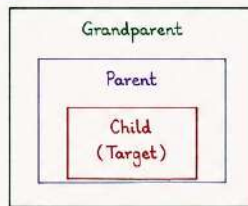
Output (Bubbling):

1. Child
- ↓
2. Parent
- ↓
3. Grandparent

Flow: Target → Parent → Grandparent → ... (Bubbles Up)

Event Propagation Order

Consider the structure:



Capturing
(Top → Down)

1. Grandparent
2. Parent
- ↓
3. Child (Target)

Bubbling
(Bottom → Up)

1. Child (Target)
2. Parent
- ↑
3. Grandparent

Complete Order (with both):

Grandparent → Parent → Child → Parent → Grandparent

2) Event Capturing

- In capturing, the event starts from the outermost element and goes down to the target element.
- It is not the default behavior. We have to use the third parameter "true" in `addEventListener()`.

```
grandparent.addEventListener("click", () =>
  console.log("Grandparent (capture)"), true);
parent.addEventListener("click", () =>
  console.log("Parent (capture)"), true);
child.addEventListener("click", () =>
  console.log("Child (capture)"), true);
```

Output (Capturing):

1. Grandparent (capture)
- ↓
2. Parent (capture)
- ↓
3. Child (capture)

Flow: Grandparent → Parent → Child (Captures Down)

- Summary:
- Event Bubbling: Target → Parent → Ancestors (bottom to top) [Default]
 - Event Capturing: Ancestors → Parent → Target (top to bottom) [Use true]
 - We can use both together by adding two listeners (one for capture, one for bubble).

10. Explain JavaScript type conversion with suitable examples.

Answer: Type conversion in JavaScript is the process of converting a value from one data type to another. It can happen automatically (**implicit**) or manually (**explicit**). JavaScript is a loosely typed language, so it converts types when needed.

I. Implicit Type Conversion (Automatic)

JavaScript automatically converts values to the required type during operations.

Example	Explanation	Result
"5" + 2	String + Number $\Rightarrow$ Number is converted to String and concatenated.	"52"
"10" - 2	String - Number $\Rightarrow$ String is converted to Number and then subtraction is done.	8
true + 1	true is converted to 1.	2
false + 5	false is converted to 0.	5
"5" == 5	String '5' is converted to Number 5 for comparison.	true
null == 0	null is converted to 0 for comparison.	true
undefined == null	Both are treated as equal in abstract equality.	true

Note: In implicit conversion, JavaScript decides the type conversion to perform based on the context.

II. Explicit Type Conversion (Manual)

We can manually convert a value from one type to another using built-in methods.

1. To String String(value)	Examples: String(123) $\rightarrow$ "123" (456).toString() $\rightarrow$ "456" true.toString() $\rightarrow$ "true"
2. To Number Number(value)	Number("123") $\rightarrow$ 123 Number("12.5") $\rightarrow$ 12.5 Number(true) $\rightarrow$ 1 Number(false) $\rightarrow$ 0 Number("abc") $\rightarrow$ NaN (Not a Number)
3. To Boolean Boolean(value)	Boolean(1) $\rightarrow$ true Boolean(0) $\rightarrow$ false Boolean("") $\rightarrow$ false Boolean("hello") $\rightarrow$ true

III. Summary of Common Conversions

Original Value	Converted To Number	Converted To String	Converted To Boolean
"25"	25	"25"	true (non-empty string)
0	0	"0"	false
1	1	"1"	true
"" (empty string)	0	""	false
null	0	"null"	false
undefined	NaN	"undefined"	false

Conclusion: Type conversion helps JavaScript perform operations between different types of data. Use implicit conversion carefully and prefer explicit conversion to avoid unexpected results.

11. How does jQuery simplify DOM manipulation compared to JavaScript?

Answer: jQuery is a JavaScript library that provides a simple and consistent API for selecting, traversing and manipulating DOM elements, handling events and creating animations.

It reduces the amount of code, handles cross-browser inconsistencies and makes DOM operations faster and easier.

Comparison with Examples:

Task	Vanilla JavaScript	jQuery
1. Select an element	<pre>let el = document.getElementById("title"); let items = document.getElementsByClassName("item");</pre>	<pre>let el = \$("#title"); let items = \$(".item");</pre>
2. Change content	<pre>el.textContent = "Hello World";</pre>	<pre>\$("#title").text("Hello World");</pre>
3. Change CSS	<pre>el.style.color = "red"; el.style.fontSize = "20px";</pre>	<pre>\$("#title").css({ color: "red", font-size: "20px" });</pre>
4. Add / remove class	<pre>el.classList.add("active"); el.classList.remove("active");</pre>	<pre>\$("#title").addClass("active"); \$("#title").removeClass("active");</pre>
5. Create new element and add to DOM	<pre>let newP = document.createElement("p"); newP.textContent = "New paragraph"; document.body.appendChild(newP);</pre>	<pre>\$("#<p>").text("New paragraph") .append("body");</pre>
6. Handle click event	<pre>el.addEventListener("click", function() { alert("Clicked!"); });</pre>	<pre>\$("#title").on("click", function() { alert("Clicked!"); });</pre>
7. Hide / show element	<pre>el.style.display = "none"; // hide el.style.display = "block"; // show</pre>	<pre>\$("#title").hide(); // hide \$("#title").show(); // show</pre>

Key Points:

- jQuery uses a simple syntax like `$(selector)` to select elements.
- It provides many built-in methods for DOM manipulation (`text()`, `html()`, `css()`, `addClass()`, etc.).
- Event handling is easy with `.on()` / `.click()` / `.submit()`, etc.
- It works consistently across different browsers.
- It shortens the code and makes it more readable and maintainable.

Conclusion: jQuery simplifies DOM manipulation by providing a concise syntax, powerful methods and cross-browser compatibility, so we can achieve more with less code compared to Vanilla JavaScript.

12. Explain jQuery selectors with suitable examples.

Answer: In jQuery, selectors are used to "find" (select) HTML elements based on their id, class, name, type, attribute, value, position, etc.

Once selected, we can apply jQuery methods and perform actions on them.

Type of Selector	Syntax	Description	Example	Selects
1. Element Selector	<code>\$("element")</code>	Selects all elements with the given tag name.	<code>\$("p")</code>	All <code><p></code> elements
2. ID Selector	<code>\$("#id")</code>	Selects the element with the given id.	<code>\$("#header")</code>	Element with id = "header"
3. Class Selector	<code>\$(".class")</code>	Selects all elements with the given class.	<code>\$(".box")</code>	All elements with class = "box"
4. Universal Selector	<code>\$("*")</code>	Selects all elements on the page.	<code>\$("*")</code>	All elements
5. Multiple Selector	<code>\$(el1, el2, ...)</code>	Selects all elements matching any of the selectors.	<code>\$(h1, p)</code>	All <code><h1></code> and <code><p></code> elements
6. Descendant Selector	<code>\$("ancestor descendant")</code>	Selects all descendants inside an ancestor.	<code>\$("div p")</code>	All <code><p></code> elements inside <code><div></code>
7. Child Selector	<code>\$("parent > child")</code>	Selects all direct children of the parent.	<code>\$("ul > li")</code>	Direct <code></code> children of <code></code>
8. Adjacent Sibling Selector	<code>\$("prev + next")</code>	Selects the next sibling that is immediately preceded by prev.	<code>\$("h2 + p")</code>	The <code><p></code> just after <code><h2></code>
9. General Sibling Selector	<code>\$("prev ~ siblings")</code>	Selects all siblings that come after prev.	<code>\$("h2 ~ p")</code>	All <code><p></code> siblings after <code><h2></code>

Examples:

- `$("p").css("color", "blue");` // Selects all `<p>` and changes text color to blue
- `$("#title").text("Hello");` // Selects element with id = title and sets its text
- `$(".box").hide();` // Hides all elements with class = box
- `$("ul > li").addClass("active");` // Adds class 'active' to all direct children `<li>` of `<ul>`
- `$("input[type='text']").val("Ayush");` // Sets value of all text input fields

Conclusion: jQuery selectors help us easily find and work with HTML elements using a simple and powerful syntax. They save time and make DOM manipulation faster and cleaner.

Attribute Selectors

Selector	Example	Selects
<code>\$("[attr]")</code>	<code>\$("[href]")</code>	Elements with the attribute 'href'
<code>\$("[attr='value']")</code>	<code>\$("[type='text']")</code>	Elements with attr having exact value
<code>\$("[attr!= 'value']")</code>	<code>\$("[type!= 'password']")</code>	Elements with attr not equal to value
<code>\$("[attr^='value']")</code>	<code>\$("[href^='https']")</code>	attr value starts with value
<code>\$("[attr\$='value']")</code>	<code>\$("[href\$='.pdf']")</code>	attr value ends with value
<code>\$("[attr*='value']")</code>	<code>\$("[href*='jquery']")</code>	attr value contains the value

Pseudo-class Selectors

Selector	Example	Selects
<code>\$(":first")</code>	<code>\$("p:first")</code>	First <code><p></code> element
<code>\$(":last")</code>	<code>\$("p:last")</code>	Last <code><p></code> element
<code>\$(":eq(n)")</code>	<code>\$("li:eq(2)")</code>	Element at index n (0-based)
<code>\$(":even")</code>	<code>\$("tr:even")</code>	Elements at even index (0, 2, 4, ...)
<code>\$(":odd")</code>	<code>\$("tr:odd")</code>	Elements at odd index (1, 3, 5, ...)

Notes:

- jQuery selectors are similar to CSS selectors.
- They are case-sensitive.
- More specific selectors are faster and better.
- You can combine selectors for more precise selection.

13. What are jQuery effects? Explain any four methods.

Answer: jQuery effects are pre-defined methods used to create animations and visual changes to HTML elements. These effects make web pages more interactive and attractive with very less code.

jQuery provides many effect methods such as `hide()`, `show()`, `toggle()`, `fadeIn()`, `fadeOut()`, `fadeToggle()`, `slideDown()`, `slideUp()`, `slideToggle()`, `animate()` etc.

Four commonly used effect methods are explained below:

① hide() - Hides the selected elements.

Syntax	Example	Description
<code>\$(selector).hide();</code> <code>[speed, callback]</code>	<code>\$("#box").hide(1000);</code> <code>// hides in 1 second</code>	Hides the matched elements by changing their display to none. The optional speed (in ms) creates a sliding effect.

② show() - Shows the hidden elements.

Syntax	Example	Description
<code>\$(selector).show();</code> <code>[speed, callback]</code>	<code>\$("#box").show(1000);</code> <code>// shows in 1 second</code>	Displays the hidden elements (display is changed from none to block/inline). Optional speed creates animation.

③ toggle() - Toggles between hide and show.

Syntax	Example	Description
<code>\$(selector).toggle();</code> <code>[speed, callback]</code>	<code>\$("#box").toggle(1000);</code> <code>// hides if visible,</code> <code>// shows if hidden</code>	If the element is visible, it will be hidden. If it is hidden, it will be shown. Optional speed creates animation.

④ fadeIn() / fadeOut() - Fades elements in or out.

Syntax	Example	Description
<code>\$(selector).fadeIn();</code> <code>[speed, callback]</code>	<code>\$("#box").fadeIn(1500);</code> <code>// fades in in 1.5 sec</code>	Gradually changes the opacity from 0 to 1 (increasing visibility).
<code>\$(selector).fadeOut();</code> <code>[speed, callback]</code>	<code>\$("#box").fadeOut(1500);</code> <code>// fades out in 1.5 sec</code>	Gradually changes the opacity from 1 to 0 (decreasing visibility).

Example in HTML/JS (using above four methods)

```
<button id="b1">Hide</button>
<button id="b2">Show</button>
<button id="b3">Toggle</button>
<button id="b4">Fade Out</button>
<button id="b5">Fade In</button>
<div id="box" style="width:120px;height:60px;background:skyblue;"
  > jQuery Effects </div>
```

```
$("#b1").click(function(){ $("#box").hide(1000); });
$("#b2").click(function(){ $("#box").show(1000); });
$("#b3").click(function(){ $("#box").toggle(1000); });
$("#b4").click(function(){ $("#box").fadeOut(1000); });
$("#b5").click(function(){ $("#box").fadeIn(1000); });
```

Key Points :

- All effect methods accept optional speed (slow, fast or milliseconds) and a callback function.
- Effects make web pages more interactive and user friendly.
- These methods can be chained together.

Conclusion: jQuery effects help to animate elements easily with simple methods. `hide()`, `show()`, `toggle()` and `fadeIn()` / `fadeOut()` are widely used to control visibility with attractive animations.

14. How does jQuery AJAX simplify asynchronous communication?

Answer: jQuery AJAX (Asynchronous JavaScript and XML) allows web pages to send and retrieve data from a server asynchronously (in the background) without reloading the entire page. jQuery simplifies AJAX by providing easy-to-use methods, consistent cross-browser support, and automatic handling of many low-level details.

How jQuery AJAX simplifies asynchronous communication?

- Simple API: Simple methods like `$.ajax()`, `$.get()`, `$.post()`, `.load()`, etc. reduce complex XMLHttpRequest code.
- Cross-browser compatibility: Works consistently across different browsers.
- Less code: Achieve the same functionality with fewer lines of code.
- Automatic handling: Takes care of creating XMLHttpRequest object, opening connection, sending request, and processing response.
- Data handling: Easily handles JSON, XML, HTML or text data.
- Callback support: Built-in callback functions for success, error, complete, beforeSend, etc.
- Synchronous or asynchronous: Asynchronous by default, keeps the user interface responsive.

Example: Using \$.get()

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery AJAX Example</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>
  <button id="btn">Load Data</button>
  <div id="result"></div>
  <script>
    // When the button is clicked,
    // a request is sent to data.txt
    // in the background and the
    // response is shown without
    // reloading the page.
    $("#btn").click(function(){
      $("#result").html("Loading...");
      $.get("data.txt", function(data){
        $("#result").html(data); // display response
      }).fail(function(){
        $("#result").html("Error while loading data.");
      });
    });
  </script>
</body>
</html>
```

Conclusion: jQuery AJAX greatly simplifies asynchronous communication by providing an easy and powerful API. It reduces code, ensures cross-browser compatibility, and helps developers build fast, dynamic web applications.

Common jQuery AJAX Methods

Method	Description	Example
<code>\$.ajax()</code>	Performs an asynchronous HTTP request with many options.	<code>\$.ajax({ url: "data.txt", method: "GET", success: function(data){...} });</code>
<code>\$.get()</code>	Requests data from the server using HTTP GET.	<code>\$.get("data.txt", function(data){ console.log(data); });</code>
<code>\$.post()</code>	Sends data to the server using HTTP POST.	<code>\$.post("save.php", {name:"Ayush"}, function(res){ console.log(res); });</code>
<code>.load()</code>	Loads data from a server and puts the returned HTML into selected elements.	<code>\$("#content").load("page.html");</code>

Equivalent AJAX using Vanilla JavaScript (for comparison)

```
var xhr = new XMLHttpRequest();
xhr.open("GET", "data.txt", true); // true → async
xhr.onload = function(){
  if(xhr.status === 200){
    document.getElementById("result").innerHTML = xhr.responseText;
  } else {
    document.getElementById("result").innerHTML = "Error while loading data.";
  }
};
xhr.onerror = function(){
  document.getElementById("result").innerHTML = "Error while loading data.";
};

document.getElementById("btn").onclick = function(){
  document.getElementById("result").innerHTML = "Loading...";
  xhr.send();
};
```

More code and manual handling required in plain JavaScript.

15. Explain jQuery Traversing methods with examples.

Answer: jQuery traversing methods allow us to "move" through the DOM tree and find (select) related elements from the currently selected element(s). They help in navigating to parent, child, sibling and other related elements.

1. Traversing Upwards (Parents)

Method	Description	Syntax	Example	Result
<code>.parent()</code>	Selects the direct parent of each matched element.	<code>\$(selector).parent()</code>	<code>\$("span").parent();</code>	Direct parent of <code></code>
<code>.parents()</code>	Selects all ancestors (parents, grand-parents, etc.)	<code>\$(selector).parents()</code>	<code>\$("span").parents();</code>	All ancestor elements
<code>.closest()</code>	Selects the first ancestor that matches the given selector.	<code>\$(selector).closest(sel)</code>	<code>\$("span").closest("ul");</code>	First ancestor matching <code></code>
<code>.parentsUntil()</code>	Selects all ancestors up to (but not including) the specified element.	<code>\$(selector).parentsUntil(sel)</code>	<code>\$("span").parentsUntil("div");</code>	Ancestors until (not including) <code><div></code>

Example HTML Structure

```
<div id="container">
  <ul id="list">
    <li class="item">Item 1 </li>
    <li class="item active">Item 2
      <span>New </span>
    </li>
    <li class="item">Item 3 </li>
  </ul>
  <p class="footer">Footer </p>
</div>
```

Key Points:

- Traversing starts from the selected element(s).
- Methods return new jQuery objects containing the matched elements.
- These methods are very useful for navigating and manipulating related parts of the DOM.

2. Traversing Downwards (Children)

Method	Description	Syntax	Example	Result
<code>.children()</code>	Selects all direct children of each matched element.	<code>\$(selector).children()</code>	<code>\$("ul").children();</code>	Direct children (<code></code> elements)
<code>.find()</code>	Selects all descendants (of any level) that match the selector.	<code>\$(selector).find(sel)</code>	<code>\$("div").find("span");</code>	All <code></code> inside <code><div></code>

3. Traversing Siblings

Method	Description	Syntax	Example	Result
<code>.siblings()</code>	Selects all siblings (of the same parent).	<code>\$(selector).siblings()</code>	<code>\$("li.active").siblings();</code>	All <code></code> siblings except the one with class <code>active</code>
<code>.next()</code>	Selects the next sibling.	<code>\$(selector).next()</code>	<code>\$("li.active").next();</code>	Immediately next sibling
<code>.prev()</code>	Selects the previous sibling.	<code>\$(selector).prev()</code>	<code>\$("li.active").prev();</code>	Immediately previous sibling
<code>.nextAll()</code>	Selects all next siblings.	<code>\$(selector).nextAll()</code>	<code>\$("li.active").nextAll();</code>	All following siblings
<code>.prevAll()</code>	Selects all previous siblings.	<code>\$(selector).prevAll()</code>	<code>\$("li.active").prevAll();</code>	All preceding siblings

Practical Examples based on above structure

Code (jQuery)	Explanation	Selects
<code>\$("span").parent();</code>	Direct parent of <code></code>	<code><li class="item active">Item 2</code>
<code>\$("span").parents();</code>	All ancestors of <code></code>	<code><li class="item active"></code> , <code><ul id="list"></code> , <code><div id="container"></code> , <code><body></code> , <code><html></code>
<code>\$("span").closest("ul");</code>	First ancestor matching <code></code>	<code><ul id="list"></code>
<code>\$("#list").children();</code>	Direct children of <code>#list</code>	All three <code></code> elements
<code>\$("#container").find("span");</code>	All <code></code> inside <code>#container</code>	<code>New</code>
<code>\$("li.active").siblings();</code>	Siblings of <code>.active</code>	First and third <code></code> elements
<code>\$("li.active").next();</code>	Next sibling of <code>.active</code>	Third <code></code> element
<code>\$("li.active").prev();</code>	Previous sibling of <code>.active</code>	First <code></code> element
<code>\$("li.active").nextAll();</code>	All next siblings	Third <code></code> element
<code>\$("li.active").prevAll();</code>	All previous siblings	First <code></code> element

Conclusion: jQuery traversing methods help us navigate through the DOM tree easily. They are essential for accessing related elements and performing dynamic operations.

16. Explain the advantages of HTML5 over previous HTML versions.

Answer: HTML5 is the latest version of HTML. It introduces many new features and improvements that make web development easier, more powerful and more efficient than previous HTML versions (HTML 4.01, XHTML).
The main advantages of HTML5 are:

Sr. No.	Advantage	Explanation	Example
1.	Simpler & Cleaner Syntax	HTML5 has a simpler and more consistent syntax. It removes unnecessary tags and attributes.	HTML4 : <code></code> , <code><center></code> , <code><u></code> (not recommended) HTML5 : Use CSS for styling and layout.
2.	New Semantic Elements	HTML5 provides meaningful elements that clearly describe the structure of a web page.	New tags : <code><header></code> , <code><nav></code> , <code><section></code> , <code><article></code> , <code><aside></code> , <code><footer></code> , <code><main></code> → Better for SEO and readability.
3.	Built-in Support for Multimedia	HTML5 supports audio and video without the need for external plugins (like Flash).	<code><audio controls> <source src="song.mp3" type="audio/mpeg"></audio></code> <code><video controls width="400" height="250">...</video></code>
4.	Graphics and Animation Support	HTML5 introduces <code><canvas></code> and SVG for drawing graphics, charts and animations.	<code><canvas id="myCanvas"></canvas></code> SVG : Scalable Vector Graphics for 2D images.
5.	Offline Web Applications	With Application Cache and Service Workers, websites can work offline or with a slow network.	Users can access cached pages even without internet.
6.	Better Forms and Validation	New input types and attributes make forms more powerful and validation easier.	<code><input type="email" required></code> <code><input type="date"></code> , <code><input type="number"></code>
7.	Geolocation Support	HTML5 allows websites to access the user's location (with permission).	<code>navigator.geolocation.getCurrentPosition(success, error);</code>
8.	Local Storage	HTML5 provides Web Storage (<code>localStorage</code> and <code>sessionStorage</code>) to store data on the client side.	<code>localStorage.setItem("name", "Ayush");</code> <code>sessionStorage.getItem("name");</code>
9.	Better Performance	HTML5 is optimized for speed and efficiency. It requires fewer resources and works faster.	HTML5 pages load faster and run smoothly on modern browsers.
10.	Cross Platform & Mobile Friendly	HTML5 is designed to work on all modern browsers, devices and platforms including mobile phones and tablets.	Same code works on Windows, Android, iOS, Mac, Linux etc.

Conclusion: HTML5 provides a more powerful, flexible and user-friendly web development experience. It improves performance, adds new features and eliminates the need for external plugins.

17. What are semantic elements in HTML5? Explain any five semantic tags.

Answer: Semantic elements in HTML5 are those elements that clearly describe the meaning or purpose of the content they contain. They make the code easier to read, improve accessibility for screen readers, help with SEO and make the structure of a web page more meaningful. HTML5 provides several semantic tags such as `<header>`, `<nav>`, `<section>`, `<article>`, `<aside>`, `<footer>`, `<main>` etc.

Five semantic tags in HTML5:

Sr.	Tag	Purpose / Description	Example	Use / Importance
1.	<code><header></code>	Represents introductory content or a set of introductory links for a section or the whole page.	<code><header></code> <code><h1> My Website </h1></code> <code><p> Welcome to my site </p></code> <code></header></code>	• Usually contains logo, site title, heading, navigation etc. • Helps search engines understand the top section of the page.
2.	<code><nav></code>	Defines a set of navigation links. It is used for menus and links to other pages or sections.	<code><nav></code> <code>Home </code> <code>About </code> <code>Contact </code> <code></nav></code>	• Clearly marks the navigation menu. • Helps assistive technologies to identify navigation areas.
3.	<code><section></code>	Represents a generic section of content on a page. Each section should have a heading.	<code><section></code> <code><h2> Our Services </h2></code> <code><p> We provide the best services. </p></code> <code></section></code>	• Used to divide the page into sections. • Improves readability and organization of content.
4.	<code><article></code>	Represents independent, self-contained content that can stand alone. (e.g., blog post, news item, article)	<code><article></code> <code><h2> How HTML5 Improves Web </h2></code> <code><p> HTML5 introduces many new features... </p></code> <code></article></code>	• Content can be distributed or reused independently. • Helps search engines identify important content.
5.	<code><footer></code>	Defines the footer for a section or the whole page. It may contain copyright, links, contact info, etc.	<code><footer></code> <code><p> © 2025 My Website. All rights reserved. </p></code> <code><p> Privacy Policy </p></code> <code></footer></code>	• Usually placed at the bottom of the page or section. • Contains useful information like copyright, terms, links, etc.

Conclusion: Semantic elements in HTML5 give meaning to the content, improve accessibility, boost SEO and make the code more clean, readable and maintainable.

18. Differentiate between <div> and semantic HTML5 elements with examples.

Answer: <div> is a generic container element with no specific meaning. Semantic HTML5 elements clearly describe their meaning or purpose to both the browser and the developer.

Basis	<div> (Generic)	Semantic HTML5 Elements
Meaning	Has no specific meaning. It is just a block-level container.	Has a specific meaning that describes the type of content it contains.
Purpose	Used only for layout and grouping elements for styling or scripting.	Used to define the structure of the content in a meaningful way.
Readability of Code	Makes the code less readable and hard to understand.	Makes the code more readable and self-explanatory.
SEO (Search Engine Optimization)	Search engines cannot understand the content inside <div> easily.	Search engines can easily understand the content which helps in better ranking.
Accessibility	Provides poor accessibility for screen readers.	Provides better accessibility for screen readers and assistive technologies.
Maintainability	Harder to maintain large projects.	Easier to maintain and update large projects.
Examples	<div>, <div class="box">, <div id="header">	<header>, <nav>, <section>, <article>, <aside>, <footer>, <main>

Example: Using <div> (Non-semantic)

Using Semantic HTML5 Elements

```
<div id="header">
  <div class="logo">My Site</div>
  <div class="menu">Home | About | Contact</div>
</div>
<div id="content">
  <div class="post">
    <div class="title">HTML5 is Awesome</div>
    <div class="text">HTML5 provides many new features.</div>
  </div>
</div>
<div id="footer">© 2025 My Site. All rights reserved.</div>
```

```
<header>
  <div class="logo">My Site</div>
  <nav>Home | About | Contact</nav>
</header>
<main>
  <article>
    <h2>HTML5 is Awesome</h2>
    <p>HTML5 provides many new features.</p>
  </article>
</main>
<footer>© 2025 My Site. All rights reserved.</footer>
```

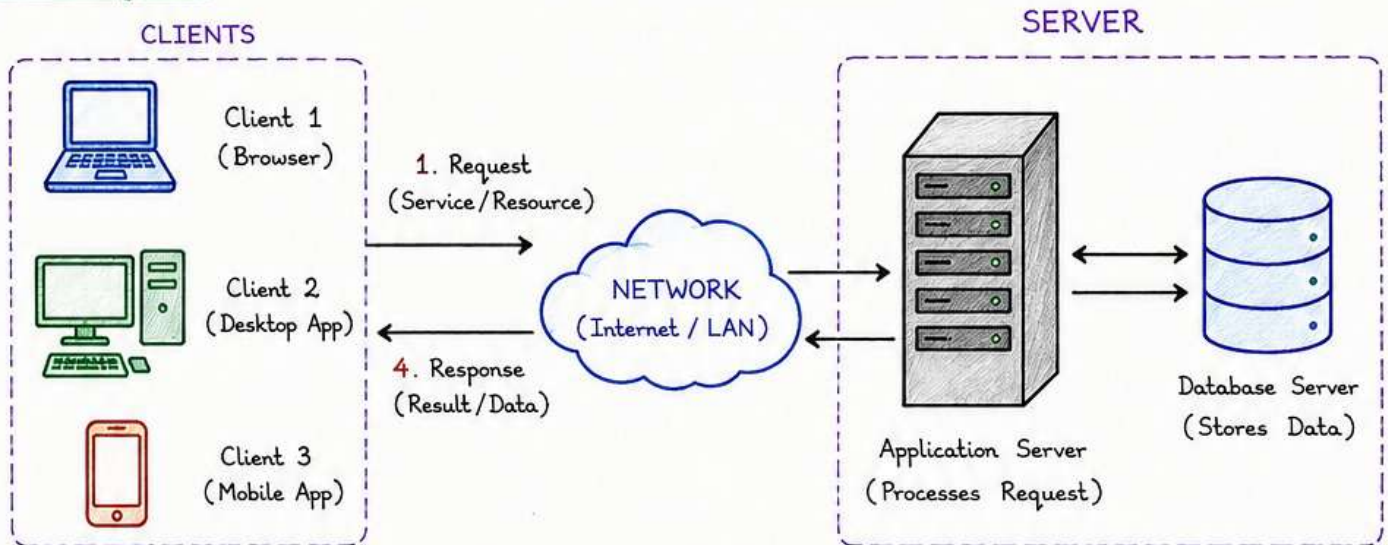
- Key Points:
- Semantic elements clearly define the role of the content.
 - They improve SEO, accessibility and code readability.
 - <div> should be used only when no semantic element is suitable.

Conclusion: While <div> is useful for grouping and styling, semantic HTML5 elements provide meaning, improve SEO, accessibility and maintainability. It is always better to use semantic elements whenever possible.

19. Explain the Client-Server Architecture with a neat diagram.

Answer: Client-Server Architecture is a distributed application architecture in which client computers (clients) request services or resources from centralized server computers (servers). The server processes the requests and sends the response back to the client.

Neat Diagram:



Components:

- **Client**: Requests services or resources from the server. (e.g., Web browser, mobile app)
- **Server**: Provides services, processes requests and manages resources. (e.g., Web server, Application server, Database server)
- **Network**: Medium that allows communication between client and server. (e.g., Internet, LAN)

Working / Flow:

1. The client sends a request to the server for a service or resource.
2. The request is transmitted over the network to the server.
3. The server receives the request, processes it, may access the database or other resources and prepares the response.
4. The server sends the response back to the client over the network.
5. The client receives the response and displays or uses the data.

Example: When you open a website in your browser:



Advantages:

- Centralized management of data and resources.
- Data security and backup are easier on the server.
- Easy maintenance and update of applications.
- Scalability - more clients can be added without affecting the server.
- Resource sharing (files, printers, services) is easy.

Disadvantages:

- If the server fails, clients cannot access resources/services.
- High cost for powerful server and maintenance.
- Requires reliable network connection.
- More complex to set up than peer-to-peer architecture.

Conclusion: Client-Server Architecture allows multiple clients to access centralized resources and services efficiently over a network, making applications scalable, secure, and easier to manage.

20. Differentiate between Client-Side Scripting and Server-Side Scripting with examples.

Answer: Client-side scripting runs in the web browser of the user, while server-side scripting runs on the web server.

Basis	Client-Side Scripting	Server-Side Scripting
1. Where it Runs	Runs on the client (web browser).	Runs on the server.
2. Examples	JavaScript, VBScript	PHP, ASP, JSP, Python, Ruby
3. Code Visibility	Code is visible to the user (view source).	Code is not visible to the user.
4. Execution	Executed by the browser.	Executed by the web server.
5. Purpose	Validates user input, improves UI, dynamic effects.	Handles database, authentication, file processing, business logic.
6. Performance	Faster response; no round trip to server.	Slower due to communication with server.
7. Security	Less secure; code can be tampered by user.	More secure; code is hidden on server.
8. Dependency	Depends on the browser (may not work in all browsers).	Independent of the browser; works on any platform.
9. Access to Resources	Cannot access server-side resources (database, files).	Can access server-side resources like database, files.
10. Usage	Used for front-end development.	Used for back-end development.

Examples:

Client-Side Scripting Example

```
<script>
  function showMsg() {
    alert("Hello, this is client-side");
  }
</script>
```

This JavaScript code runs in the browser and shows a message.

Server-Side Scripting Example

```
<?php
  $name = $_POST['name'];
  echo "Welcome, ". $name;
?>
```

This PHP code runs on the server, processes the data and sends the result to the browser.

Conclusion: Client-side scripting makes the web pages interactive, while server-side scripting handles the data and business logic securely.

21. Explain the purpose of the `addEventListener()` method with an example. How does it differ from inline event handling?

Answer: The `addEventListener()` method is used to attach an event handler to an HTML element. It allows JavaScript to respond to user actions such as clicks, key presses, and mouse movements without writing event code directly in HTML.

Syntax:

`element.addEventListener("event", function);`

Example:

```
<button id="btn">Click Me</button>

<script>
  document.getElementById("btn").
    addEventListener("click", function() {
      alert("Button Clicked!");
    });
</script>
```

Difference from Inline Event Handling:

- `addEventListener()` keeps JavaScript separate from HTML, making code cleaner and easier to maintain.
- It allows multiple event handlers to be attached to the same element.
- Inline event handling writes JavaScript directly inside HTML tags (e.g., `onclick`), which is less organized and not recommended for large applications.

22. What is Event Delegation in JavaScript? Why is it useful?

Answer: Event Delegation is a technique in JavaScript where an event listener is attached to a parent element instead of attaching separate listeners to each child element. It works because events bubble from the child element to the parent.

Example:

```
<ul id="list">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
<script>
  document.getElementById("list")
    .addEventListener("click", function(event) {
      if (event.target.tagName === "LI") {
        alert(event.target.textContent);
      }
    });
</script>
```

event object

↑

element that triggered the event

Why is it useful?

- **Reduces** the number of event listeners.
- **Improves** performance and memory usage.
- **Automatically** works for dynamically added child elements.
- Makes the code **simpler** and **easier** to maintain.

23. Explain different String methods in JavaScript with examples.

Answer: String methods are built-in JavaScript functions used to manipulate and process string values.

1. **length** - Returns the number of characters in a string.

Example:

```
let str = "Hello";  
console.log(str.length);
```

 // Output: 5

2. **toUpperCase()** - Converts all characters to uppercase.

Example:

```
let str = "hello";  
console.log(str.toUpperCase());
```

 // Output: HELLO

3. **toLowerCase()** - Converts all characters to lowercase.

Example:

```
let str = "HELLO";  
console.log(str.toLowerCase());
```

 // Output: hello

4. **charAt()** - Returns the character at a specified index.

Example:

```
let str = "Hello";  
console.log(str.charAt(1));
```

 // Output: e

5. **indexOf()** - Returns the position of the first occurrence of a specified character or substring.

Example:

```
let str = "JavaScript";  
console.log(str.indexOf("S"));
```

 // Output: 4

6. **substring()** - Returns a part of a string between two indexes.

Example:

```
let str = "JavaScript";  
console.log(str.substring(0, 4));
```

 // Output: Java

7. **replace()** - Replaces a specified value with another value.

Example:

```
let str = "Hello World";  
console.log(str.replace("World", "JavaScript"));
```

 // Output: Hello JavaScript

8. **split()** - Splits a string into an array based on a separator.

Example:

```
let str = "A,B,C";  
console.log(str.split(","));
```

 // Output: ["A","B","C"]

24. Explain different Math object methods in JavaScript.

Answer: The Math object in JavaScript provides built-in methods to perform mathematical calculations. It does not require creating an object.

1. **Math.abs()** - Returns the absolute (positive) value of a number.

Example: `console.log(Math.abs(-10));` // Output: 10

2. **Math.sqrt()** - Returns the square root of a number.

Example: `console.log(Math.sqrt(25));` // Output: 5

3. **Math.pow()** - Returns the value of a number raised to a given power.

Example: `console.log(Math.pow(2, 3));` // Output: 8

4. **Math.max()** - Returns the largest value from the given numbers.

Example: `console.log(Math.max(10, 20, 30));` // Output: 30

5. **Math.min()** - Returns the smallest value from the given numbers.

Example: `console.log(Math.min(10, 20, 30));` // Output: 10

6. **Math.round()** - Rounds a number to the nearest integer.

Example: `console.log(Math.round(4.6));` // Output: 5

7. **Math.floor()** - Rounds a number down to the nearest integer.

Example: `console.log(Math.floor(4.9));` // Output: 4

8. **Math.ceil()** - Rounds a number up to the nearest integer.

Example: `console.log(Math.ceil(4.1));` // Output: 5

9. **Math.random()** - Returns a random number between 0 (inclusive) and 1 (exclusive).

Example: `console.log(Math.random());` // Output: 0.3257... (any value between 0 and 1)

25. Explain Date Object methods in JavaScript.

Answer: The Date object in JavaScript is used to work with dates and time. It provides methods to get or set different parts of a date.

1. `getDate()` - Returns the day of the month (1-31).

Example:

```
let d = new Date();  
console.log(d.getDate());
```

 // Output: (e.g., 26)

2. `getMonth()` - Returns the month (0-11), where January is 0.

Example:

```
let d = new Date();  
console.log(d.getMonth());
```

 // Output: (e.g., 5 for June)

3. `getFullYear()` - Returns the four-digit year.

Example:

```
let d = new Date();  
console.log(d.getFullYear());
```

 // Output: (e.g., 2025)

4. `getDay()` - Returns the day of the week (0-6), where Sunday is 0.

Example:

```
let d = new Date();  
console.log(d.getDay());
```

 // Output: (e.g., 1 for Monday)

5. `getHours()` - Returns the current hour (0-23).

Example:

```
let d = new Date();  
console.log(d.getHours());
```

 // Output: (e.g., 14)

6. `getMinutes()` - Returns the current minutes (0-59).

Example:

```
let d = new Date();  
console.log(d.getMinutes());
```

 // Output: (e.g., 37)

7. `getSeconds()` - Returns the current seconds (0-59).

Example:

```
let d = new Date();  
console.log(d.getSeconds());
```

 // Output: (e.g., 45)

8. `getTime()` - Returns the number of milliseconds since 1 January 1970.

Example:

```
let d = new Date();  
console.log(d.getTime());
```

 // Output: (e.g., 1716723456789)

26. What is Browser Object Model (BOM)?

Explain its commonly used objects.

Answer: The Browser Object Model (BOM) allows JavaScript to interact with the web browser. It provides objects and methods to control browser windows, navigate pages, display dialogs, and access browser information.

Commonly Used BOM Objects

1. window Object

The top-level object representing the browser window. It provides methods such as `alert()`, `prompt()`, and `setTimeout()`.

Example: `window.alert("Welcome!");`

2. document Object

Represents the HTML document and is used to access and manipulate webpage elements.

Example: `document.getElementById("msg").innerHTML = "Hello";`

3. navigator Object

Provides information about the browser, such as its name and version.

Example: `console.log(navigator.userAgent);`

4. location Object

Provides information about the current URL and allows page redirection or reloading.

Example: `location.href = "home.html";`

5. history Object

Used to navigate the browser's history.

Example: `history.back();`

6. screen Object

Provides information about the user's screen, such as width and height.

Example: `console.log(screen.width);`

27. Explain JavaScript Error Handling using try, catch, throw, and finally.

Answer: JavaScript Error Handling is used to detect and handle runtime errors so that the program does not stop unexpectedly. It uses the try, catch, throw, and finally statements.

1. try

The try block contains the code that may generate an error.

Example:

```
try {  
    console.log(a);  
}
```

2. catch

The catch block handles the error if one occurs in the try block.

Example:

```
try {  
    console.log(a);  
}  
catch(error) {  
    console.log("Error:", error.message);  
}
```

3. throw

The throw statement is used to create and throw a custom error.

Example:

```
let age = 15;  
if (age < 18) {  
    throw "Not eligible to vote";  
}
```

4. finally

The finally block always executes whether an error occurs or not.

Example:

```
try {  
    console.log("Hello");  
}  
catch(error) {  
    console.log(error);  
}  
finally {  
    console.log("Program Ended");  
}
```


28. What is Strict Mode in JavaScript? Explain its advantages.

Answer: Strict Mode is a special mode in JavaScript that helps write safer and error-free code. It is enabled by adding the statement `"use strict"`; at the beginning of a script or function.

Syntax

```
"use strict";
```

Example

```
"use strict";
```

```
x = 10;    // Error: x is not declared
```

Advantages of Strict Mode

1. **Prevents Undeclared Variables** - Variables must be declared before use.
2. **Detects Common Coding Errors** - Helps identify mistakes during development.
3. **Improves Code Security** - Prevents unsafe JavaScript practices.
4. **Makes Debugging Easier** - Errors are reported clearly, making them easier to fix.
5. **Encourages Better Coding Practices** - Promotes cleaner and more reliable code.

29. Explain the advantages of using jQuery over plain JavaScript.

Answer: jQuery is a lightweight JavaScript library that simplifies web development by providing easy-to-use methods for DOM manipulation, event handling, animations, and AJAX.

Advantages of jQuery

1. **Simple Syntax** - Performs tasks with fewer lines of code than plain JavaScript.
2. **Easy DOM Manipulation** - Makes selecting and modifying HTML elements simple.
3. **Cross-Browser Compatibility** - Works consistently across different web browsers.
4. **Easy Event Handling** - Simplifies handling events like clicks, key presses, and mouse actions.
5. **Built-in Effects and Animations** - Provides methods such as `hide()`, `show()`, `fadeIn()`, and `slideDown()`.
6. **Simplified AJAX** - Makes asynchronous communication with the server easier using methods like `$.ajax()`.

Example

Plain JavaScript

```
document.getElementById("msg").innerHTML = "Hello";
```

jQuery

```
$("#msg").html("Hello");
```


30. Explain the role of HTML5 APIs in modern web development.

Answer: HTML5 APIs provide built-in functionalities that allow web applications to perform advanced tasks without requiring external plugins. They help developers create faster, interactive, and feature-rich web applications.

Role of HTML5 APIs

1. **Geolocation API** - Retrieves the user's current location.
2. **Web Storage API** - Stores data in the browser using `localStorage` and `sessionStorage`.
3. **Canvas API** - Draws graphics, animations, and games directly in the browser.
4. **Drag and Drop API** - Allows users to drag and drop elements on a webpage.
5. **Web Workers API** - Runs JavaScript in the background without affecting the user interface.
6. **WebSocket API** - Enables real-time communication between the browser and the server.

Example

```
navigator.geolocation.getCurrentPosition(function(position) {  
    console.log(position.coords.latitude);  
});
```


Button Click

1) With onclick

```
<button onclick="show()">Click Me</button>

<script>
function show()
{
    alert("Button Clicked");
}
</script>
```

2) With addEventListener

```
<button id="btn">Click</button>
<script>
document.getElementById("btn")
    .addEventListener("click", function()
    {
        alert("Clicked");
    });
</script>
```

Functions

```
function add(a,b)
{
    return a + b;
}
console.log(add(5,3)); // 8
```

Loops

1) For Loop

```
for(let i = 1; i <= 5; i++)
{
    console.log(i);
}

// Output: 1 2 3 4 5
```

2) While Loop

```
let i = 1;
while(i <= 5)
{
    console.log(i);
    i++;
}

// Output: 1 2 3 4 5
```

Type Conversion

Syntax:- NewType(value);

```
Number(value);
String(value);
Boolean(value);
BigInt(value);
```

String Methods

name.length
name.toUpperCase()

Method	Use Case
length	Get string length
toUpperCase()	Convert to uppercase
toLowerCase()	Convert to lowercase
charAt(index)	Get character at index
indexOf(text)	Find first position of text
lastIndexOf(text)	Find last position of text
includes(text)	Check if text exists
startsWith(text)	Check starting text
endsWith(text)	Check ending text
substring(start, end)	Extract part of string
slice(start, end)	Extract part (supports negative index)
replace(old, new)	Replace text
replaceAll(old, new)	Replace all occurrences
trim()	Remove spaces from both ends
split(separator)	Convert string into array
concat(str)	Join two strings

DOM Manipulation

- Change HTML
document.getElementById("title").innerHTML = "Welcome";
- Change Text
document.getElementById("title").textContent = "Welcome";
- Change Text Color
document.getElementById("title").style.color = "red";
- Change Background Color
document.getElementById("title").style.backgroundColor = "yellow";
- Change Font Size
document.getElementById("title").style.fontSize = "30px";
- Hide Element
document.getElementById("title").style.display = "none";
- Show Element
document.getElementById("title").style.display = "block";
- Change Input Value
document.getElementById("name").value = "Ayush";
- Get Input Value
let x = document.getElementById("name").value;
- Change Image
document.getElementById("photo").src = "image.jpg";
- Change Link
document.getElementById("link").href = "https://example.com";
- Disable Button
document.getElementById("btn").disabled = true;
- Enable Button
document.getElementById("btn").disabled = false;

Math Object

Method	Use Case
Math.abs(x)	Absolute (positive) value
Math.ceil(x)	Round up
Math.floor(x)	Round down
Math.round(x)	Round to nearest integer
Math.trunc(x)	Remove decimal part
Math.sqrt(x)	Square root
Math.pow(x, y)	Power (x^y)
Math.max(a, b, ...)	Largest value
Math.min(a, b, ...)	Smallest value
Math.random()	Random number (0 to <1)
Math.PI	Value of π (Property, no ())

Object

```
// 1. Create Object
let student = {
    name: "Ayush",
    age: 20,

    // 4. Method
    greet: function() {
        console.log("Hello " + this.name);
    }
};

// 2. Read Property
console.log(student.name); // Ayush
console.log(student.age); // 20

// 3. Update Property
student.age = 21;
console.log(student.age); // 21

// 4. Call Method
student.greet(); // Hello Ayush
```

jQuery

1) Selectors

JavaScript	jQuery
document.getElementById("title")	\$("#title")
document.getElementsByClassName("box")	\$(".box")
document.getElementsByTagName("p")	\$("p")
document.querySelectorAll("*")	\$("*")
this	\$(this)

2) Effects

Syntax:- \$("#box").show();

Method	Use Case
hide()	Hide an element
show()	Display a hidden element
toggle()	Switch between hide and show
fadeIn()	Show element with fade animation

Method	Use Case
fadeOut()	Hide element with fade animation
slideUp()	Hide element by sliding upward
slideDown()	Show element by sliding downward

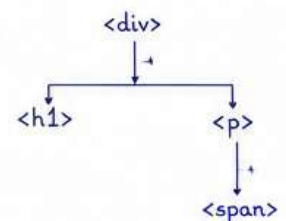
3) AJAX Call

```
$.ajax({
    url: "data.txt",

    success: function(result){
        console.log(result);
    }
});
```

4) Traversing

One Family Diagram (Remember this!)



Method	Meaning
parent()	Go ↑ to parent
children()	Go ↓ to direct children
find()	Search inside descendants
siblings()	Same parent
next()	Next sibling
prev()	Previous sibling

```
$("#text").parent(); // Gets parent
$(".box").children(); // Gets children
$("#box").find("span"); // Finds span inside
$("#title").siblings(); // Gets siblings
```